

# Natural Language Processing (NLP)

Introduction to NLP. Author: David Thébault

# NLP – Definition and goal

**Natural Language Processing (NLP)** is a subfield of artificial intelligence (AI) that focuses on the interaction between computers and humans through natural language.

**The goal of NLP** is to enable computers to understand, interpret, and generate human language in a way that is both meaningful and contextually appropriate.

# NLP - Common tasks and applications

**Translation**

**Summarization**

**Question answering**

**Speech recognition**

**Classification**

**Assisted Writing**

**...**

# NLP - Difficulties

**Ambiguity:** “I saw the banker with the telescope.”

**Polysemy:** “The bank robbery was quick.”

**Homonymy:** “The bass was too loud.”

**Synonymy:** “The cat is on the couch.”

**Context dependency:** “I am going to the bank.”

**Idioms:** “It’s raining cats and dogs.”

**Sarcasm:** “Oh great, it’s raining again.”

**Negation:** “I did not say I would come.”

**Coreference Resolution:** “Mary said she would come.”

**Named Entity Recognition:** “Paris is the capital of France.”

# NLP - Fundamentals

**Basic Linguistics:** Phonetics, morphology, syntax, semantics, pragmatics.

**Language Structure:** Sentences, words, morphemes, phonemes.

**Ambiguity and Context:** Issues of lexical and syntactic ambiguity, the role of context.

# NLP – Workflow Cycle example

- 1.Data Collection:** Download a corpus of tweets.
- 2.Data Preprocessing:** Clean the tweets, tokenize, remove stop words.
- 3.Exploratory Data Analysis (EDA):** Visualize the most frequent words.
- 4.Feature Selection:** Use TF-IDF to represent the tweets.
- 5.Modelling:** Train a sentiment classification model using an SVM.
- 6.Evaluation:** Measure the precision and recall of the model.
- 7.Optimization and Tuning:** Adjust the hyperparameters of the SVM.
- 8.Deployment:** Integrate the model into a web application.
- 9.Maintenance and Updating:** Periodically retrain the model with new tweets.

# NLP – Install SpaCy

*# SpaCy is an open-source NLP library for advanced text processing in Python.*

*# Install + download a model*

*# version 1*

```
!pip install spacy
```

```
!python -m spacy download en_core_web_sm
```

*# version 2*

```
!conda create --name env python=3.11
```

```
!conda activate env
```

```
!conda uninstall -y numpy h5py spacy
```

```
!conda install numpy h5py spacy
```

```
!conda uninstall -y numpy h5py spacy
```

```
!conda install --upgrade numpy h5py
```

```
!conda install ipykernel
```

```
!python -m spacy download en_core_web_sm
```

```
!python -m spacy info
```

# NLP

## **PREPROCESSING**



# NLP – Tokenization

*# Tokenization*

```
import spacy
nlp = spacy.load('en_core_web_sm')

sentence = "He didn't want to pay $20 for the book."
doc = nlp(sentence)
print(doc)
print([t.text for t in doc]) # iterate over the Doc object
print(doc[0:3]) # Slicing
print([(t.text, t.i) for t in doc])
```

```
He didn't want to pay $20 for the book.
['He', 'did', "n't", 'want', 'to', 'pay', '$', '20', 'for', 'the', 'book', '.']
He didn't
[('He', 0), ('did', 1), ("n't", 2), ('want', 3), ('to', 4), ('pay', 5), ('$', 6), ('20', 7),
('for', 8), ('the', 9), ('book', 10), ('.', 11)]
```

# NLP – Lower Casing

```
# Case Folding
```

```
print([t.lower_ for t in doc])
```

```
['he', 'did', "n't", 'want', 'to', 'pay', '$', '20', 'for', 'the', 'book', '.']
```

# NLP – Stop words

# Stop words: removing words which occur frequently but carry little information

```
import spacy
nlp = spacy.load('en_core_web_sm')
# Print tokens and indicate if they are stop words

sentence = "I saw the movie last night. I was not amused."
doc = nlp(sentence)

filtered_sentence = ' '.join([token.text for token in doc if not token.is_stop])
print(filtered_sentence)

saw movie night . amused .
```

# NLP – Stemming

# Stemming: removing words suffixes and sometimes prefixes

# The goal is to reduce words at the same base form.

# Typically done through an algorithmn the most famous of which is Porter's algorithm:

# Banking -> Bank

# Banks -> Bank

# Analysis -> Analysi

# Because of this last example, we don't use Stemming. + *Not possible with spaCy*

# NLP – Lemmatization

*# Lemmatization*

```
import spacy
nlp = spacy.load('en_core_web_sm')
doc = nlp("Did, Done, Doing")
[(t.text, t.lemma_) for t in doc]
```

```
[('Did', 'do'), (',', ','), ('Done', 'Done'), (',', ','), ('Doing', 'do')]
```

# NLP – POS Tagging

# Part-of-Speech Tagging: Assign each word in a sentence its POS tagger (VERB, NOUN,...)

```
import spacy
```

```
nlp = spacy.load('en_core_web_sm')
```

*# Exemple of sentence*

```
sentence = "John is a watching an old movie at a cinema."
```

*# Creating a Doc object that contains linguistic annotations for the text.*

```
doc = nlp(sentence)
```

*# POS (course-grained) tags thanks to pos\_ attribute*

```
[(t.text, t.pos_) for t in doc]
```

```
[('John', 'PROPN'),  
 ('is', 'AUX'),  
 ('a', 'DET'),  
 ('watching', 'VERB'),  
 ('an', 'DET'),  
 ('old', 'ADJ'),  
 ('movie', 'NOUN'),  
 ('at', 'ADP'),  
 ('a', 'DET'),  
 ('cinema', 'NOUN'),  
 ('.', 'PUNCT')]
```

*# to get a description for a POS tag, use spacy explain() method*

```
spacy.explain('PROPN')
```

```
'proper noun'
```

# NLP – NER

# Name Entity Recognition (NER): Tagging of proper names, locations, ('real world') entities.

# Example of sentence

s = "Volkswagen is developing an electric sedan which could potentially come to America next fall."

doc = nlp(s)

# To access named other we use here the spacy attribute ent\_type\_  
# other ways to make it are possible with Spacy.

[(t.text, t.ent\_type\_) for t in doc]

```
[('Volkswagen', 'ORG'),  
 ('is', ''),  
 ('developing', ''),  
 ('an', ''),  
 ('electric', ''),  
 ('sedan', ''),  
 ('which', ''),  
 ('could', ''),  
 ('potentially', ''),  
 ('come', ''),  
 ('to', ''),  
 ('America', 'GPE'),  
 ('next', 'DATE'),  
 ('fall', 'DATE'),  
 ('.', '')]
```

# to get a description for a POS tag, use spacy explain() method

spacy.explain(ORG')

Companies, agencies, institutions, etc.

# NLP – Parsing

# Parsing consists in determining the syntactic structure of a sentence.

*# Dependency Parsing*

```
import spacy
```

*# Load the english language model of SpaCy*

```
nlp = spacy.load("en_core_web_sm")
```

*# Sentence to analyse*

```
sentence = "She enrolled in the course at the university."
```

*# Analyze the sentence*

```
doc = nlp(sentence)
```

*# Display the dependency relations*

```
for token in doc:
```

```
    print(f"{token.text} ({token.dep_}) <-- {token.head.text}")
```

```
She (nsubj) <-- enrolled
enrolled (ROOT) <-- enrolled
in (prep) <-- enrolled
the (det) <-- course
course (pobj) <-- in
at (prep) <-- enrolled
the (det) <-- university
university (pobj) <-- at
. (punct) <-- enrolled
```

*# Let's visualize a dependency parse*

```
displacy.render(doc, style='dep', jupyter=True)
```



# NLP – Matcher

```
# Patterns are expressed as an ordered sequence.
# Here, we're looking to match occurrences starting with a 'book' string followed by
# a determiner (DET) POS tag such as "the","and" , then a noun POS tag.
# The OP key marks the match as optional in some way.

# Here, the DET POS (marked with '?') will match 0 or 1 times (i.e. the determiner is optional), and
# the NOUN POS (marked with '+') will match 1 or more times (i.e., at least one noun is required).
# See this link for more information.
# https://spacy.io/usage/rule-based-matching#quantifiers

pattern = [
    {'TEXT': 'book'},
    {'POS': 'DET', 'OP': '?'},
    {'POS': 'NOUN', 'OP': '+'}
]

# So, the pattern will match sequences that start with the word "book",
# optionally followed by a determiner, and then followed by one or more nouns.
# We give our pattern a label and pass it to the matcher.
matcher.add('USER_INTENT', [pattern])

# Run the matcher over the doc.
matches = matcher(doc)

# For each match, the matcher returns a tuple specifying a match id, start,
# and end of the match.
print("Matches: ", [doc[start:end].text for match_id, start, end in matches])
Matches: ['book a hotel', 'book a hotel room']
```

# NLP – Preprocessing with SpaCy

```
import spacy
from sklearn.datasets import fetch_20newsgroups

# Download the *train* dataset without headers, footers, and quotes to make the problem more challenging.
train_corpus = fetch_20newsgroups(subset='train', remove=('headers', 'footers', 'quotes'))

# We don't need named entity recognition nor parsing. Removing them will speed up processing.
nlp = spacy.load('en_core_web_sm', disable= ['ner', 'parser'])

def spacy_tokenizer(doc):
    return [t.lemma_.lower() for t in nlp(doc) if \
            len(t) > 2 and \
            not t.is_punct and \
            not t.is_space and \
            not t.is_stop and \
            t.is_alpha]

def preprocess_text(text):
    tokens = spacy_tokenizer(text)
    return "|".join(tokens)

preprocessed_train_corpus = [preprocess_text(post) for post in train_corpus.data]
```

# NLP – Preprocessing with SpaCy

*# We don't need named entity recognition nor parsing. Removing them will speed up processing.*

```
nlp = spacy.load('en_core_web_sm', disable= ['ner', 'parser'])
```

```
preprocessed_train_corpus = [preprocess_text(post) for post in train_corpus.data]
```

```
from sklearn.model_selection import train_test_split
```

```
train_data, val_data, train_labels, val_labels = train_test_split(preprocessed_train_corpus,  
                                                                    train_corpus.target,  
                                                                    train_size=0.85,  
                                                                    random_state=1)
```

# NLP

## **VECTORIZATION**

Bag of Words (BoW): Simple representation based on word frequency.

# NLP – Bag Of Words (BOW)

**\*\*Describing documents by word occurrences\*\***

# Basic idea: if two documents share the same vocabulary, the more likely they belong to the same class.

# This is called bag-of-words because of the lack of order of the words, no grammar considering, no syntax.

# We will see:

# – Frequency bag of words: when we count the occurrences of the word document per document

# – Binary bag of words: 1 if the word is present in the document, 0 elsewhere.

# NLP – Bag Of Words (BOW)

## # Frequency bag of words

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# A corpus of sentences.
```

```
corpus = [  
    "Red Bull drops hint on F1 engine.",  
    "Honda exits F1, leaving F1 partner Red Bull.",  
    "Hamilton eyes record eighth F1 title.",  
    "Aston Martin announces sponsor."  
]
```

```
vectorizer = CountVectorizer()
```

```
bow = vectorizer.fit_transform(corpus)
```

```
print(bow)
```

Corpus has been vectorized into a matrix. Each row corresponds to a document while each column corresponds to a token. In our Corpus, there are 4 documents and 20 different words, so the matrix will have 4 rows and 20 columns.

Values are equal to the number of times the word occur in the document.

| Coords  | Values |
|---------|--------|
| (0, 17) | 1      |
| (0, 2)  | 1      |
| (0, 3)  | 1      |
| (0, 10) | 1      |
| (0, 14) | 1      |
| (0, 8)  | 1      |
| (0, 5)  | 1      |
| (1, 17) | 1      |
| (1, 2)  | 1      |
| (1, 8)  | 2      |
| (1, 11) | 1      |
| (1, 6)  | 1      |
| (1, 12) | 1      |
| (1, 15) | 1      |
| (2, 8)  | 1      |
| (2, 9)  | 1      |
| (2, 7)  | 1      |
| (2, 16) | 1      |
| (2, 4)  | 1      |
| (2, 19) | 1      |
| (3, 1)  | 1      |
| (3, 13) | 1      |
| (3, 0)  | 1      |
| (3, 18) | 1      |

# NLP – Bag Of Words (BOW)

## # Frequency bag of words

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# A corpus of sentences.
```

```
corpus = [  
    "Red Bull drops hint on F1 engine.",  
    "Honda exits F1, leaving F1 partner Red Bull.",  
    "Hamilton eyes record eighth F1 title.",  
    "Aston Martin announces sponsor."  
]
```

```
vectorizer = CountVectorizer()
```

```
bow = vectorizer.fit_transform(corpus)
```

```
print(bow.toarray())
```

```
# View features (tokens).
```

```
print(vectorizer.get_feature_names_out())
```

```
# View vocabulary dictionary.
```

```
print(vectorizer.vocabulary_)
```

As we already said, `fit_transform()` returns a matrix where each row represents a document, and each column represents a token



```
[[0 0 1 1 0 1 0 0 1 0 1 0 0 0 1 0 0 1 0 0]  
 [0 0 1 0 0 0 1 0 2 0 0 1 1 0 0 1 0 1 0 0]  
 [0 0 0 0 1 0 0 1 1 1 0 0 0 0 0 0 1 0 0 1]  
 [1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0]]
```

# NLP – Bag Of Words (BOW)

# Binary bag of words

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# A corpus of sentences.
```

```
corpus = [  
    "Red Bull drops hint on F1 engine.",  
    "Honda exits F1, leaving F1 partner Red Bull.",  
    "Hamilton eyes record eighth F1 title.",  
    "Aston Martin announces sponsor."  
]
```

```
vectorizer = CountVectorizer(tokenizer=spacy_tokenizer, lowercase=False, binary=True)  
bow = vectorizer.fit_transform(corpus)  
print(bow.toarray())
```

This time with the parameter `binary=True` we have a 1 if the word is in the document and 0 otherwise



```
[[0 1 1 0 0 0 1 0 1 0 1 0 0 1 0 1 0 0 0 0]  
 [0 1 1 0 1 0 1 0 0 0 0 1 0 0 1 0 1 0 0 0]  
 [0 0 1 1 0 0 0 0 0 1 0 0 1 0 0 0 0 1 0 1]  
 [1 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0]]
```



# NLP – Bag Of Words (BOW)

```
# Binary bag of words
```

```
import pandas as pd
```

```
# Display the result into a DataFrame:
```

```
pd.DataFrame(bow.toarray(), columns = vectorizer.get_feature_names_out())
```

|   | Aston | Bull | F1 | Hamilton | Honda | Martin | Red | ... |
|---|-------|------|----|----------|-------|--------|-----|-----|
| 0 | 0     | 1    | 1  | 0        | 0     | 0      | 1   | 0   |
| 1 | 0     | 1    | 1  | 0        | 1     | 0      | 1   | 0   |
| 2 | 0     | 0    | 1  | 1        | 0     | 0      | 0   | 0   |
| 3 | 1     | 0    | 0  | 0        | 0     | 1      | 0   | 1   |

# NLP – Bag Of Words (BOW)

# Cosine similarity between documents a and b

# Now that documents have been transformed into vectors we can compute a similarity between documents.

$$\text{cosine}(a, b) = \frac{a \cdot b}{\sqrt{||a|| \times ||b||}} = \cos(\theta)$$

With:

$$a \cdot b = \sum a_i \times b_i$$

$$||a|| = a \cdot a = \sum a_i^2$$

$$\theta = \widehat{(a, b)}$$

When a and b points in the same direction i.e. are similar,  $\cos(0) = 1 = -\cos(\pi)$

When a and b are orthogonal i.e. dissimilar,  $\cos\left(\frac{\pi}{2}\right) = \cos\left(-\frac{\pi}{2}\right) = 0$

Because high frequency words will lead to larger dot products... To avoid this, we normalize by vector length  $\sqrt{||v||} = \sqrt{\sum v_i^2}$  known also as the  $L^2$  norm or Euclidean norm.

# NLP – Bag Of Words (BOW)

```
# Cosine similarity between documents a and b

from sklearn.metrics.pairwise import cosine_similarity

# cosine_similarity can take either array-likes or sparse matrices.

print(cosine_similarity(bow))

[[1.          0.42857143 0.15430335 0.          ]
 [0.42857143 1.          0.15430335 0.          ]
 [0.15430335 0.15430335 1.          0.          ]
 [0.          0.          0.          1.          ]]

# The first and the second document are the more similar.
```

# NLP – Bag Of Words (BOW)

Drawbacks of BOW:

- Does not capture similarity between synonyms.
- No way to handle Out-of-Vocabulary (OOV) words.
- Creates sparse vectors which can be inefficient
- Word order information is lost.

# NLP – N-grams

## Definition:

Chunks of continuous tokens.

2-grams or bigram has two tokens per chunk.

3-grams or trigram has three tokens per chunk.

...

Example of tokenization into bigram:

"Barcelona beats Chelsea" => ["Barcelona beats", "beats Chelsea"]

`CountVectorizer()` includes sparser `ngram_range` parameter to generate different n-grams.

N-grams range is specified using a **minimum** and **maximum** range.

By default, n-grams range is set to (1, 1) which generates unigrams.

Setting it to (1, 2) generates both unigrams and bigrams.

Unigram or 1-gram are equivalent to binary bags of words.

When we use N-grams it is recommended to keep stop words or at least keep the negation because the bi-gram « Not good » will have a different meaning of the unigram « good ». It is one advantage of N-grams in comparison with binary BOW. N-grams capture the sequence of words, which allows to keep a part of the local context. This can improve the precision of the NLP model for sentiment analysis, text classification or NER. N-grams are also flexible. You can easily get in the same time unigram, bigram, or tri-grams just fixing the parameter `ngram_range = (1, 3)`.

On the other hand, N-grams increase the complexity and the size of the vocabulary. N-grams are sparser than BOW.

# NLP – N-grams

```
# INTRO
# A corpus of sentences.
corpus = [
    "Red Bull drops hint on F1 engine.",
    "Honda exits F1, leaving F1 partner Red Bull.",
    "Hamilton eyes record eighth F1 title.",
    "Aston Martin announces sponsor."
]

vectorizer = CountVectorizer(tokenizer=spacy_tokenizer, lowercase=False, binary=True, ngram_range=(1,2), stop_words=None) # unigram & bigrams
bigrams = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print('Number of features: {}'.format(len(vectorizer.get_feature_names_out())))
print(vectorizer.vocabulary_)

['Aston' 'Aston Martin' 'Bull' 'Bull drops' 'F1' 'F1 engine' 'F1 leaving'
 'F1 partner' 'F1 title' 'Hamilton' 'Hamilton eyes' 'Honda' 'Honda exits'
 'Martin' 'Martin announces' 'Red' 'Red Bull' 'announces'
 'announces sponsor' 'drops' 'drops hint' 'eighth' 'eighth F1' 'engine'
 'exits' 'exits F1' 'eyes' 'eyes record' 'hint' 'hint on' 'leaving'
 'leaving F1' 'on' 'on F1' 'partner' 'partner Red' 'record'
 'record eighth' 'sponsor' 'title']
Number of features: 40
{'Red': 15, 'Bull': 2, 'drops': 19, 'hint': 28, 'on': 32, 'F1': 4, 'engine': 23, 'Red Bull': 16, 'Bull drops': 3, 'drops hint': 20, 'hint on': 29, 'on F1': 33, 'F1 engine': 5, 'Honda': 11, 'exits': 24, 'leaving': 30, 'partner': 34, 'Honda exits': 12, 'exits F1': 25, 'F1 leaving': 6, 'leaving F1': 31, 'F1 partner': 7, 'partner Red': 35, 'Hamilton': 9, 'eyes': 26, 'record': 36, 'eighth': 21, 'title': 39, 'Hamilton eyes': 10, 'eyes record': 27, 'record eighth': 37, 'eighth F1': 22, 'F1 title': 8, 'Aston': 0, 'Martin': 13, 'announces': 17, 'sponsor': 38, 'Aston Martin': 1, 'Martin announces': 14, 'announces sponsor': 18}
```

# NLP – N-grams

```
# INTRO
# A corpus of sentences.
corpus = [
    "Red Bull drops hint on F1 engine.",
    "Honda exits F1, leaving F1 partner Red Bull.",
    "Hamilton eyes record eighth F1 title.",
    "Aston Martin announces sponsor."
]

vectorizer = CountVectorizer(tokenizer=spacy_tokenizer, lowercase=False, binary=True, ngram_range=(1,2), stop_words=None) # unigram & bigrams
bigrams = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
print('Number of features: {}'.format(len(vectorizer.get_feature_names_out())))
print(vectorizer.vocabulary_)

['Aston' 'Aston Martin' 'Bull' 'Bull drops' 'F1' 'F1 engine' 'F1 leaving'
 'F1 partner' 'F1 title' 'Hamilton' 'Hamilton eyes' 'Honda' 'Honda exits'
 'Martin' 'Martin announces' 'Red' 'Red Bull' 'announces'
 'announces sponsor' 'drops' 'drops hint' 'eighth' 'eighth F1' 'engine'
 'exits' 'exits F1' 'eyes' 'eyes record' 'hint' 'hint on' 'leaving'
 'leaving F1' 'on' 'on F1' 'partner' 'partner Red' 'record'
 'record eighth' 'sponsor' 'title']
Number of features: 40
{'Red': 15, 'Bull': 2, 'drops': 19, 'hint': 28, 'on': 32, 'F1': 4, 'engine': 23, 'Red Bull': 16, 'Bull drops': 3, 'drops hint': 20, 'hint on': 29, 'on F1': 33, 'F1 engine': 5, 'Honda': 11, 'exits': 24, 'leaving': 30, 'partner': 34, 'Honda exits': 12, 'exits F1': 25, 'F1 leaving': 6, 'leaving F1': 31, 'F1 partner': 7, 'partner Red': 35, 'Hamilton': 9, 'eyes': 26, 'record': 36, 'eighth': 21, 'title': 39, 'Hamilton eyes': 10, 'eyes record': 27, 'record eighth': 37, 'eighth F1': 22, 'F1 title': 8, 'Aston': 0, 'Martin': 13, 'announces': 17, 'sponsor': 38, 'Aston Martin': 1, 'Martin announces': 14, 'announces sponsor': 18}
```

# NLP – N-grams

```
# USE CASE N-GRAMS IN FRENCH (1/2)
```

```
# Load the french model
```

```
!python -m spacy download fr_core_news_sm
```

```
import spacy
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import classification_report
```

```
# Load the spaCy french model
```

```
nlp = spacy.load("fr_core_news_sm")
```

```
# Example of comments data
```

```
avis = [
```

```
    "J'adore ce produit, il est incroyable!",
```

```
    "Ce produit est terrible, je le déteste.",
```

```
    "Le produit est correct, mais pas exceptionnel.",
```

```
]
```

```
# Corresponding sentiment labels
```

```
sentiments = ["positif", "négatif", "neutre"]
```



# NLP – N-grams

```
# USE CASE N-GRAMS (2/2)

# pre-processing of the comments
def preprocess(text):
    doc = nlp(text)
    return ' '.join([token.lemma_ for token in doc if not token.is_stop and not token.is_punct])

avis_pretraites = [preprocess(av) for av in avis]

print(f"avis_pretraites: {avis_pretraites} \n")
# Génération de N-grammes
vectorizer = CountVectorizer(ngram_range=(1, 3)) # Unigrammes, bigrammes et trigrammes
X = vectorizer.fit_transform(avis_pretraites)
print(f"vectorizer.get_feature_names_out() :\n {vectorizer.get_feature_names_out()} \n")
print(f"X: \n {X.toarray()} \n")
# Division des données en ensembles d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, sentiments, test_size=0.2, random_state=42)

# Entraînement du modèle SVM
model = SVC(kernel='linear')
model.fit(X_train, y_train)

# Prédiction et évaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

# NLP – Term Frequency-Inverse Document Frequency

TF-IDF is another bag-of-words technique still popular today.

With the previous BOW techniques all the words have the same importance. Even removing stop words, you can have highly frequent words but not informative.

For instance, in a corpus of sports you can have a high frequency of "play" and "games" but can be not informative.

We will use relative frequency instead:  
How often a token appears in one document versus how often it appears in other documents.

If a word occurs frequently in one or few documents but not in the rest, it's likely meaningful for those documents and should be considered more "important".

In the other hand if a word occurs frequently across the corpus, then it should be considered less "important" and meaningful.

# NLP – Term Frequency-Inverse Document Frequency

## Term Frequency

$$tf(t, d) = \log_{10}(\text{Count}\{t \text{ in } d\} + 1)$$

With  $t$  the *term*,  $d$  the *document*.

The document could simply have much more words, so we use the logarithm to dampen the effect.

## Inverse Document Frequency

$$idf(t, D) = \log_{10}\left(\frac{N}{n_t}\right)$$

With  $D$  the corpus,  $N$  the number of documents,  $n_t$  the number of documents where  $t$  appears in.

If  $t$  appears in every document,  $N = n_t$ ,  $\log_{10}\left(\frac{N}{n_t}\right) = \log(1) = 0$

# NLP – Term Frequency-Inverse Document Frequency

Term Frequency – Inverse Document Frequency:

$$w_{t,d} = tf(t,d) \times idf(t,D) = \log_{10}(\text{Count}\{t \text{ in } d\} + 1) \times \log_{10}\left(\frac{N}{n_t}\right)$$

**The more frequently a term appears in a document  $\times$  the fewer times it appears in other documents, the higher the TF-IDF value is.**

TF-IDF was so popular that many variations were created: e.g. the default TF-IDF vectorizer in `scikit-learn`:

$$w_{t,d} = tf(t,d) \times idf(t,D) = \log_{10}(\#\{t \text{ in } d\} + 1) \times \left(\ln\left(\frac{N + 1}{n_t + 1}\right) + 1\right)$$

# NLP – Term Frequency-Inverse Document Frequency

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Use of TfidfVectorizer.
vectorizer = TfidfVectorizer(tokenizer=spacy_tokenizer, stop_words='english')

# We transform the corpus of documents into a matrix of characteristics TF-IDF
features = vectorizer.fit_transform(corpus.data)
```

# NLP – Term Frequency-Inverse Document Frequency

```
# Querying the documents
```

```
# View the 5 first row of the columns 'lunar' & 'orbit' in the matrix of TF-IDF:
```

```
pd.DataFrame(features.toarray(), columns = vectorizer.get_feature_names_out()).head()[['lunar', 'orbit']]
```

```
# Transform the query into a TF-IDF vector.
```

```
query = ["lunar orbit"]
```

```
query_tfidf = vectorizer.transform(query)
```

# NLP – Term Frequency-Inverse Document Frequency

*# USE CASE: Find the document the most similar to a request:*

*# Transform the query into a TF-IDF vector.*

```
query = ["lunar orbit"]
```

```
query_tfidf = vectorizer.transform(query)
```

*# The request "lunar orbit" is first tokenized and lemmatized using the `spacy_tokenizer` function. The vocabulary and these weights are stored in the `TfidfVectorizer`.*

*# When you use `vectorizer.transform(query)`, the vectorizer uses the learned vocabulary to transform the query into a TF-IDF vector.*

*# For each token in the query, the vectorizer checks if the token exists in the learned vocabulary. If it does, it uses the learned TF-IDF weights for that token.*

*# If not, the token is ignored. How TF-IDF is computed on the tokens of the query:  $TF = 1$  for both "lunar" and "orbit".*

*# The IDF is the  $\log(N/n_t)$  for "lunar" and for "orbit" corresponding to the corpus. If "lunar" is in 10 documents of the corpus out of 1000 documents in the corpus:  $IDF = \log(1000/10)$ .*

```
from sklearn.metrics.pairwise import cosine_similarity
```

*# Compute the cosine similarity between the query and the first 100 documents*

```
similarities = cosine_similarity(query_tfidf, features[:100])
```

*# Find the indices of the documents the most similar*

```
most_similar_docs_indices = similarities.argsort()[0][-5:][::-1]
```

*# Display the document the most similar*

```
print("Most similar documents:")
```

```
for idx in most_similar_docs_indices:
```

```
print(f"Document {idx}: {corpus.data[idx][:200]}") # Display the 200 first characters of each document
```

# NLP – Text Data Representation

TF-IDF: Term Frequency-Inverse Document Frequency.

Word Embeddings: Vector representations of words (Word2Vec, GloVe, FastText).

Contextual Embeddings: Contextual representations (BERT, ELMo).



# NLP – NLP Techniques

- Language Models: N-grams, Hidden Markov Models (HMM).
- Syntactic Parsing: Parsing, syntax trees.
- Semantic Analysis: Anaphora resolution, sense disambiguation.
- Machine Translation: Statistical and neural translation models.
- Sentiment Analysis: Classification of sentiments in text.

# NLP – Machine Learning for NLP

- Supervised Algorithms: Logistic regression, SVM, decision trees.
- Unsupervised Algorithms: Clustering, LDA (Latent Dirichlet Allocation).
- Neural Networks: Multilayer perceptrons, RNN (Recurrent Neural Networks), LSTM (Long Short-Term Memory), GRU (Gated Recurrent Units).
- Transformers: Transformer architecture, BERT, T5.

# NLP – Evaluation of NLP Models

- Evaluation Metrics: Precision, recall, F1-score, BLEU, ROUGE.
- Cross-Validation: Techniques for cross-validation to evaluate models.
- Model Comparison: Comparing the performance of different models.

# NLP – Advanced NLP Applications

- Chatbots and Virtual Assistants: Design and implementation of chatbots.
- Text Generation: Text generation models, creative applications.
- Information Extraction: Relation extraction, event extraction.
- Discourse Analysis: Analysis of discourse structure, coherence, and cohesion.

# NLP – Challenges and Future Directions

- Current Challenges: Ambiguity, context, low-resource languages.
- Ethics and Bias: Ethical issues, bias in data and models.
- Future Perspectives: Current and future trends in NLP.

# NLP – Practical Projects and Case Studies

- Course Projects: Practical projects to apply learned concepts.
- Case Studies: Real-world case studies to illustrate NLP applications.